Binary Search

> → **Key Idea**: Search sorted array by repeatedly dividing search space in half

> • **When to use**: Sorted arrays, search spaces, optimization problems, finding boundaries

> ⊕ **Complexity**: Time O(log n), Space O(1)

> ⚠ **Common Pitfall**: Use `mid = left + (right - left) / 2` to avoid integer overflow

Overview

Visual Example

Binary Search Process

```
Array: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]  Target: 7
Step 1: [1, 3, 5, 7, 9, |11|, 13, 15, 17, 19]  mid=11 > 7, go left
Step 2: [1, 3, |5|, 7, 9] 11, 13, 15, 17, 19   mid=5 < 7, go right
Step 3:  1, 3, 5, |7|, 9  11, 13, 15, 17, 19   mid=7 = 7, FOUND!

>> Each step eliminates half the search space: 10 → 5 → 2 → 1
```

Search Space Visualization

```
Lower Bound vs Upper Bound:

Array: [2, 2, 2, 5, 5, 8, 8, 8, 8]  Target: 5

Lower Bound (first ≥ target):    Upper Bound (first > target):
[2, 2, 2, |5, 5|, 8, 8, 8, 8]   [2, 2, 2, 5, 5, |8|, 8, 8, 8]
         ↑                                  ↑
      index 3                            index 5

Range of target 5: [3, 4]
```

Binary search is a search algorithm that finds the position of a target value within a sorted array. It compares the target value to the middle element and eliminates half of the array from consideration.

The key insight is that by comparing the target with the middle element, we can determine which half of the array to search next, effectively halving the search space in each iteration.

Core Implementation

Standard Binary Search

```
public int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2; // Avoid overflow

        if (nums[mid] == target) {
            return mid; // Target found
        } else if (nums[mid] < target) {
            left = mid + 1; // Target is in right half
        } else {
            right = mid - 1; // Target is in left half
        }
    }

    return -1; // Target not found
}
```

Common Problem Patterns

1. Range Search - First and Last Occurrence

Problem: Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value. If target is not found in the array, return [-1, -1]. You must write an algorithm with O(log n) runtime complexity.

Example:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Explanation: Target 8 is found at indices 3 and 4.

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
Explanation: Target 6 is not found.
```

LeetCode 34 - Find First and Last Position of Element in Sorted Array

```
public int[] searchRange(int[] nums, int target) {
    int first = lowerBound(nums, target);
```

```
        // Check if target exists
        if (first == nums.length || nums[first] != target) {
            return new int[]{-1, -1};
        }

        int last = upperBound(nums, target) - 1;
        return new int[]{first, last};
    }

    // Alternative approach using modified binary search
    public int[] searchRangeAlternative(int[] nums, int target) {
        int[] result = {-1, -1};

        // Find first occurrence
        result[0] = findFirst(nums, target);
        if (result[0] == -1) return result;

        // Find last occurrence
        result[1] = findLast(nums, target);

        return result;
    }

    private int findFirst(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                result = mid;
                right = mid - 1; // Continue searching left
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return result;
    }

    private int findLast(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                result = mid;
                left = mid + 1; // Continue searching right
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
```

```
        }

        return result;
    }
```

2. Search in Rotated Sorted Array

Problem: There is an integer array nums sorted in ascending order (with distinct values). Prior to being passed to your function, nums is possibly rotated at an unknown pivot index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be rotated at pivot index 3 and become [4,5,6,7,0,1,2]. Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.

Example:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

```
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) return mid;

            // Determine which half is sorted
            if (nums[left] <= nums[mid]) {
                // Left half is sorted
                if (target >= nums[left] && target < nums[mid]) {
                    right = mid - 1; // Target in left half
                } else {
                    left = mid + 1;   // Target in right half
                }
            } else {
                // Right half is sorted
                if (target > nums[mid] && target <= nums[right]) {
                    left = mid + 1;   // Target in right half
                } else {
                    right = mid - 1;  // Target in left half
                }
            }
        }
```

```
        return -1;
    }
```

3. Find Minimum in Rotated Sorted Array

Problem: Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array nums = [0,1,2,4,5,6,7] might become [4,5,6,7,0,1,2] if it was rotated 4 times. Given the sorted rotated array nums of unique elements, return the minimum element of this array.

Example:

```
Input: nums = [3,4,5,1,2]
Output: 1
Explanation: The original array was [1,2,3,4,5] rotated 3 times.

Input: nums = [4,5,6,7,0,1,2]
Output: 0
Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
```

```
    public int findMin(int[] nums) {
        int left = 0, right = nums.length - 1;

        while (left < right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] > nums[right]) {
                // Minimum is in right half
                left = mid + 1;
            } else {
                // Minimum is in left half (including mid)
                right = mid;
            }
        }

        return nums[left];
    }
```

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics

- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Standard Binary Search	O(log n)	O(1)
Lower/Upper Bound	O(log n)	O(1)
Search Range	O(log n)	O(1)
Binary Search on Answer	O(log(max-min) × f(n))	O(1)

Where f(n) is the complexity of the feasibility check function.

← Previous: Union-Find | Back to Main Guide | Next: Two Pointers →